

Plan de test - HealthAI Coach (MSPR3 / TPRE601)

Ce document decrit la strategie de test de la plateforme HealthAI Coach. Chaque service possede sa propre chaine d'integration continue (GitHub Actions) qui teste le code avant de publier une image Docker sur GHCR. Le perimetre couvre les 8 services de la plateforme. L'application mobile (mini reseau social) est prise en charge par un collegue et reste hors perimetre.

La reference de la chaine CI/CD est `MSPR-Deploy/CICD.md`. L'architecture detaillee est dans `MSPR/CLAUDE.md`.

1. Strategie d'ensemble

La strategie suit une pyramide de tests classique, adaptee a la techno de chaque service :

- **Tests unitaires** : logique metier isolee (cleaners ETL, scoring fitness, validators, services). Rapides, sans dependance externe.
- **Tests d'integration** : verifient le comportement avec une dependance reelle (PostgreSQL, MongoDB) lancee en service ephemere dans la CI.
- **Tests end-to-end (e2e)** : uniquement sur le Front, via Cypress, sur l'application bildee et servie.
- **Validation** : pour les services sans code applicatif (BDD, MongoDB), on valide l'application sequentielle des migrations SQL et l'init des collections / index.

Le lint et la verification de types completent les tests selon la techno (ruff, oxlint + eslint + vue-tsc, `node --check`).

Deux services imposent un **seuil de couverture de 80 %** : l'API (JaCoCo, lignes et branches) et Reco-Fitness (pytest-cov). Les autres services Python mesurent la couverture sans seuil bloquant en CI a ce stade.

2. Declenchement des tests (CI)

Tous les workflows se declenchent sur `push` et `pull_request` vers la branche par default du depot, ainsi que sur les tags Git `vX.Y.Z`. La branche par default varie selon le depot (`main` ou `master`), ce qui est repris dans le tableau ci-dessous.

Service	Branche(s)	Declencheurs
API	main	push, PR, tags v*. *.* *
Auth	main	push, PR, tags v*. *.* *
AI-Nutrition	master	push, PR, tags v*. *.* *
Reco-Fitness	master, main	push, PR, tags v*. *.* *
ETL	master, main	push, PR, tags v*. *.* *
Front	main	push, PR, tags v*. *.* *
BDD	main	push, PR, tags v*. *.* *
MongoDB	master, main	push, PR, tags v*. *.* *

Blocage du build : la publication de l'image Docker (`docker-publish.yml` , appele en `workflow_call`) a pour dependance (`needs`) les jobs de test et de lint. Elle ne s'execute que sur `push` (jamais sur les pull requests) et seulement si tous les jobs precedents passent. Concretement, un test ou un lint en echec stoppe le pipeline avant la publication : aucune image n'est pousse sur GHCR. Sur les pull requests, les tests tournent et conditionnent la fusion, mais aucune image n'est publiee.

3. Strategie par service

3.1 API (Spring Boot 4, Java 21)

- **Type** : tests unitaires JUnit (Maven).
- **CI** (`.github/workflows/ci.yml`) : job `test` lance `./mvnw test` sous JDK 21 (Temurin).
- **Couverture** : JaCoCo, seuil **80 % sur les lignes ET les branches** (`jacoco-maven-plugin` , regle `jacoco-check` , element `BUNDLE`). Le rapport est genere a la phase `test` ; le controle du seuil s'execute a la phase `verify` .
- **SonarCloud** : une etape conditionnelle lance `./mvnw -B verify sonar:sonar` (donc declenche aussi le controle JaCoCo). Elle ne s'active que si le secret GitHub `SONAR_TOKEN` est present, sinon elle est ignoree sans casser le pipeline.
- **Local** :

```
cd MSPR-API
./mvnw test           # tests + rapport JaCoCo (target/site/jacoco/)
./mvnw verify        # tests + controle du seuil 80 %
```

3.2 Auth (Bun, Hono, better-auth)

- **Type** : tests unitaires via le runner integre de Bun.
- **CI** : installation des deps avec `bun install --frozen-lockfile` , puis `bun test` .
- **Couverture** : pas de seuil impose. Suites presentes dans `src/__tests__/` (routes, entitlements, subscriptions, admin, templates email).

- **Local :**

```
cd MSPR-AUTH
bun install
bun test
```

3.3 AI-Nutrition (FastAPI, Python 3.12)

- **Type** : lint + tests unitaires avec couverture (pytest).
- **CI** : deux jobs.
 - `lint` : `ruff check .` (ruff 0.8.4).
 - `test` : checkout du depot MSPR-DB pour disposer des migrations, installation des deps (index PyTorch CPU), puis `pytest -m "not slow"`. La variable `MIGRATIONS_DIR` pointe vers `MSPR-DB/migrations`. Les rapports de couverture HTML (`htmlcov/`) et XML (`coverage.xml`) sont publiés en artefacts.
- **Couverture** : mesurée par `pytest-cov` (`--cov=app`), cible 80 %. L'enforcement strict (`--cov-fail-under`) n'est pas active dans la configuration a ce stade.
- **Local** :

```
cd MSPR-AI-Nutrition
ruff check .
pytest -m "not slow" # exclut les tests reseau / inference reelle
```

3.4 Reco-Fitness (FastAPI, Python 3.12)

- **Type** : lint + tests unitaires avec couverture (pytest-cov). Marqueurs `unit`, `integration`, `slow`.
- **CI** : deux jobs.
 - `lint` : `ruff check --config reco-fitness/ruff.toml reco-fitness/app reco-fitness/tests` (ruff 0.8.4).
 - `test` : installation des deps puis `pytest -m "not slow and not integration" -v`.
- **Couverture** : `pytest-cov` (`--cov=app`), seuil **80 %** retenu pour ce service. Les tests `integration` (containers PostgreSQL + MongoDB éphémères) et `slow` sont désélectionnés en CI ; ils se lancent en local. La configuration `pytest.ini` fixe `--cov-fail-under=0`, l'objectif 80 % est vérifié sur le rapport `term-missing`.
- **Local** :

```
cd MSPR-Reco-Fitness/reco-fitness
ruff check app/ tests/
pytest # tout (nécessite Docker pour integration/s
pytest -m "not slow and not integration" # rapide, comme la CI
```

3.5 ETL (Python 3.12)

- **Type** : lint + format + tests unitaires + tests d'integration PostgreSQL.
- **CI** : trois jobs.
 - `lint` : `ruff check` et `ruff format --check` (ruff 0.15.6, config `etl/pyproject.toml`).
 - `test` : `python -m pytest tests/ -m "not integration" -v` (unitaires : cleaners, extractors).
 - `test-integration` : lance un service PostgreSQL 16, puis `python -m pytest tests/ -m integration -v` avec les variables `DB_HOST/PORT/NAME/USER/PASSWORD` pointant vers le container.
- **Couverture** : pas de seuil impose.
- **Local** :

```
cd MSPR-ETL/etl
ruff check .
ruff format --check .
pytest -m "not integration"      # unitaires
pytest -m integration           # necessite un PostgreSQL accessible (cf. variables DB
```

3.6 Front (Vue 3, Vite, TypeScript)

- **Type** : lint + verification de types + tests unitaires (Vitest) + tests e2e (Cypress).
- **CI** : quatre jobs (Node 22).
 - `lint` : `npm run lint` (oxlint + eslint).
 - `type-check` : `npm run type-check` (vue-tsc).
 - `test` : `npm run test:unit -- --run` (Vitest).
 - `e2e` : build en mode test (`npm run build:test`), installation du binaire Cypress, puis `start-server-and-test` qui sert l'app (`vite preview --port 4173`) et lance `cypress run`. Les captures d'ecran sont publiees en artefact en cas d'echec. Ce job depend de `test`.
- **Couverture** : pas de seuil impose.
- **Local** :

```
cd MSPR-FRONT
npm ci
npm run lint
npm run type-check
npm run test:unit -- --run      # Vitest
npm run test:e2e                # build:test + Cypress
```

3.7 BDD (PostgreSQL 17, migrations SQL)

- **Type** : validation des migrations (pas de code applicatif a tester).

- **CI** : job `validate` . Lance un service PostgreSQL 17 (`postgres:17-alpine`), installe le client `psql` , puis applique sequentiellement les migrations trieées (`migrations/V*__*.sql` , tri `sort -V`) avec `psql -v ON_ERROR_STOP=1` . Une erreur SQL stoppe le pipeline.
- **Local** :

```
# Le plus simple : laisser le container appliquer les migrations au demarrage
cd MSPR-DB
docker compose up -d db
# Verification manuelle equivalente a la CI (PostgreSQL accessible) :
for f in $(ls migrations/V*__*.sql | sort -V); do psql -v ON_ERROR_STOP=1 -f "$f"; d
```

3.8 MongoDB (init collections / index)

- **Type** : verification de syntaxe + validation de l'init Mongo.
- **CI** : job `validate` . `node --check` sur chaque script `init/*.js` , puis demarrage d'un container `mongo:7-jammy` avec les scripts montes en `docker-entrypoint-initdb.d` . Un `mongosh` verifie ensuite la presence des collections attendues (`user_fitness_profiles` , `workout_programs` , `recommendation_history`) et de l'index `user_id_unique` . Une collection ou un index manquant fait echouer le job.
- **Local** :

```
cd MSPR-MongoDB
for f in init/*.js; do node --check "$f"; done      # syntaxe
docker compose up -d                               # applique l'init au demarrage
```

4. Tableau recapitulatif : type de test x service

Service	Unitaire	Integration	E2E	Lint / Types	Validation	Couverture
API	JUnit (<code>mvnw test</code>)	-	-	-	-	JaCoCo, 80 % lignes + branches
Auth	<code>bun test</code>	-	-	-	-	mesuree, sans seuil
AI-Nutrition	<code>pytest -m "not slow"</code>	-	-	ruff	-	pytest-cov, cible 80 %
Reco-Fitness	<code>pytest -m "not slow and not integration"</code>	<code>pytest -m integration</code> (PG + Mongo, local)	-	ruff	-	pytest-cov, 80 %
ETL	<code>pytest -m "not integration"</code>	<code>pytest -m integration</code> (PostgreSQL)	-	ruff + format	-	mesuree, sans seuil
Front	Vitest	-	Cypress	oxlint + eslint + vue-tsc	-	mesuree, sans seuil
BDD	-	-	-	-	migrations SQL (psql)	-
MongoDB	-	-	-	<code>node -check</code>	collections + index (mongosh)	-

Legende : - = non applicable a ce service.

5. Synthèse des seuils de couverture

Service	Outil	Seuil	Portee	Enforcement
API	JaCoCo	80 %	lignes + branches	bloquant (<code>mvnw verify</code> , regle <code>jacoco-check</code>)
Reco-Fitness	pytest-cov	80 %	lignes (<code>app</code>)	objectif, verifie sur le rapport
AI-Nutrition	pytest-cov	80 % (cible)	lignes (<code>app</code>)	mesure, pas d'enforcement strict en CI
Auth, ETL, Front	<code>bun test</code> / <code>pytest</code> / Vitest	-	-	mesure ou tests seuls, pas de seuil

Les services sans code applicatif (BDD, MongoDB) ne sont pas concernes par la couverture : ils sont valides par l'execution réelle des migrations et de l'init.